EECS3311 Stage 1 - AI Study and Learning Agent

Task 1 – Define Your Agent Project

Task 1.1 – Project Description

AI Study and Learning Agent

AI Study and Learning Agent is a Java-based application designed to help university students study and understand their course materials. Students can upload course materials as PDF or TXT files, such as lecture slides, readings, and notes. The agent can summarize uploaded materials, extract important concepts, answer questions based on course content, generate study plans, create flashcards and quizzes, grade quizzes, explain answers, identify weak topics, and automatically adjust quiz difficulty based on the student's previous performance.

Problem Addressed

University students often have a large amount of course material to review, especially when preparing for tests and exams. Reading lecture slides, notes, and readings can take a significant amount of time, and students may not always know which topics they understand well and which topics require more attention.

Students also spend time creating flashcards, finding practice questions, and organizing study schedules. This system brings these study activities together in one application and uses AI to help students review and practice their own course material more efficiently.

An AI agent is useful for this project because it can handle different study tasks together. For example, it can find information from uploaded materials, generate practice questions, and use previous quiz results to suggest what the student should focus on next.

Target Users

The main target users are undergraduate and graduate university students who have multiple courses and want help organizing, reviewing, and practicing their course material. The system provides both a

graphical user interface (MainGUI) and a command-line interface (CLI), allowing students to access the major functionality through either interface.

Agent Capabilities

Reasoning:
The agent uses Google Gemini 2.5 Flash-Lite to understand course material, answer student questions, explain concepts, and generate study content.

Planning:
The agent creates personalized study plans using the exam date, available study time, course concepts, and the student's weak topics.

Tool Use:
The agent uses software components to process documents, retrieve relevant information, communicate with Gemini, analyze quiz results, and store course information.

Retrieval:
Uploaded documents are divided into smaller sections and indexed. When the student asks a question, the system retrieves the sections that are most relevant to the question and supplies them to Gemini as context.

Memory:
The system stores course materials, concepts, flashcards, study plans, quiz results, and previous question answering interactions. This information can be used in later study activities.

Decision Making:
The system analyzes previous quiz results to determine an appropriate difficulty level when the student selects adaptive quiz mode.

Multi Step Execution:
Some features involve multiple steps. For example, when generating a study plan, the system first collects the course concepts and weak topics. It then sends this information to Gemini, processes the response, creates a StudyPlan, and saves it.

AI Model

The system will use Google Gemini 2.5 Flash-Lite as its main LLM. Gemini will be used for document summarization, concept extraction, question answering, study-plan generation, flashcard generation, quiz generation, short-answer evaluation, and answer explanations.

The application will not communicate directly with Gemini from every feature. Instead, LLMFacade provides a simple interface to the AI subsystem. PromptBuilder constructs prompts step by step for different AI tasks. GeminiProxy controls access to the Gemini service and handles behaviour such as

retries and caching when appropriate. The proxy communicates with the shared GeminiClient, which performs the actual API request.

Task 1.2 – Feature Specification

F01 – Course Creation

Description:
Users can create courses with a course name, subject area, and target exam date. Course materials and study activities are organized under the selected course.

User Interaction (GUI):
The user selects New Course, enters the course information, and clicks Create.

Input:

courseName : String
subject : String
examDate : LocalDate

Output:
A new Course object is created, stored, and displayed.

AI Involvement:
Deterministic. No LLM call is required.

Expected Workflow:

1. The user selects a New Course.
2. The course creation form is displayed.
3. The user enters the course name, subject, and exam date.
4. CourseController.createCourse(...) receives the request.
5. CourseService.createCourse(...) validates the data and creates a Course.
6. StudyRepository.saveCourse(course) stores the course.
7. The new course is displayed.

Error/Alternative Cases:

- Empty course names are rejected.
- Duplicate course names display an error.
- Past exam dates are rejected.
- If storage fails, the user is informed.

F02 – Upload Learning Materials

Description:
Users can upload PDF or TXT course materials. The system extracts and stores their text and indexes it for later retrieval.

User Interaction (GUI):
The user opens a course and selects Upload Material.

Input:

file : File
courseId : String

Output:
A stored and indexed Material.

AI Involvement:
Deterministic.

Expected Workflow:

1. The student selects a PDF or TXT file.
2. CourseController.uploadMaterial(...) receives the request.
3. CourseService.uploadMaterial(...) validates the file.
4. CourseService.extractText(file) extracts its text.
5. A Material is created.
6. StudyRepository.saveMaterial(material) stores it.
7. RAGEngine.index(material) divides the material into Chunk objects.
8. The material is displayed.

Error/Alternative Cases:

- Unsupported files are rejected.
- Oversized files are rejected.
- Password protected PDFs cannot be processed.
- PDFs without readable text are rejected.
- Failed extraction prevents the file from being stored.

F03 – Document Summarization

Description:
The agent generates a summary of an uploaded document.

User Interaction (GUI):
The student selects a material and clicks Summarize.

Input:
material : Material

Output:
A generated summary.

AI Involvement:
AI based.

Expected Workflow:

1. StudyController.summarize(material) is called.
2. LLMFacade.summarize(text) begins the AI operation.
3. PromptBuilder adds summarization instructions and the material text.
4. PromptBuilder.build() creates a Prompt.
5. GeminiProxy.generate(prompt) sends the request through GeminiClient.
6. Gemini returns the summary.
7. The summary is attached to the Material.
8. StudyRepository.saveMaterial(material) saves the update.
9. The summary is displayed.

Error/Alternative Cases:

- Materials without text cannot be summarized.
- Large documents can be processed in sections.
- Failed requests are retried once.
- If the retry fails, an error is displayed.

F04 – Concept Extraction

Description:
The agent identifies important concepts and provides a short definition for each.

User Interaction (GUI):
The student selects a material and clicks Extract Concepts.

Input:
material : Material

Output:
List<Concept>

AI Involvement:
AI based.

Expected Workflow:

1. StudyController.extractConcepts(material) is called.
2. LLMFacade.extractConcepts(text) begins processing.
3. PromptBuilder creates the concept-extraction prompt.
4. GeminiProxy.generate(prompt) sends it to Gemini.
5. Gemini returns structured concept information.
6. ResponseParser.parseConcepts(response) creates Concept objects.
7. StudyRepository.saveConcepts(...) stores them.
8. The concepts are displayed.

Error/Alternative Cases:

- Materials with insufficient content are rejected.
- Invalid structured responses are retried.
- If no meaningful concepts are found, the user is informed.

F05 – Question Answering Over Materials

Description:
Students can ask questions about uploaded course materials.

User Interaction (GUI):
The user enters a question in the Ask section.

Input:

question : String
courseId : String

Output:
An answer grounded in the uploaded material.

AI Involvement:
Hybrid.

Expected Workflow:

1. The student submits a question.
2. RAGEngine.retrieve(question, courseId) retrieves relevant Chunk objects.
3. LLMFacade.answer(question, chunks) is called.
4. PromptBuilder adds the question and the retrieved context.
5. A Prompt is created.
6. GeminiProxy sends the prompt to Gemini.
7. Gemini returns a grounded answer.
8. The answer and source information are displayed.

9. StudyRepository.saveLearningSession(...) records the interaction.

Error/Alternative Cases:

- Empty questions are rejected.
- If no relevant material is found, the system does not answer from general knowledge.
- Failed Gemini requests are retried once.

F06 – Personalized Study Plan Generation

Description:
The agent creates a study plan based on the exam date, available time, course concepts, and weak topics.

User Interaction (GUI):
The student selects Generate Study Plan and enters available daily study time.

Input:

course : Course
dailyHours : int

System generated information:

concepts : List<Concept>
weakTopics : Map<String, Double>

Output:
A StudyPlan containing StudySession objects.

AI Involvement:
Hybrid.

Expected Workflow:

1. The student requests a study plan.
2. ProgressAnalyzer.getWeakTopics(courseId) analyzes previous quiz results.
3. StudyRepository.getConcepts(courseId) retrieves course concepts.
4. LLMFacade.generateStudyPlan(...) is called.
5. PromptBuilder adds the exam date, study time, concepts, and weak topics.
6. GeminiProxy sends the prompt to Gemini.
7. ResponseParser.parseStudyPlan() creates study-session data.
8. LearningActivityFactory.createStudyPlan() creates the final StudyPlan.
9. StudyRepository.saveStudyPlan() stores it.
10. The plan is displayed.

Error/Alternative Cases:

- Invalid study time is rejected.
- Past exam dates are rejected.
- Without course material, the student is asked to upload material first.
- Without quiz history, a general plan is generated.

F07 – Flashcard Generation

Description:
The agent generates flashcards from material or extracted concepts.

User Interaction (GUI):
The student selects a source and number of cards.

Input:

source : Material or List<Concept>
count : int

Output:
List<Flashcard>

AI Involvement:
AI-based.

Expected Workflow:

1. StudyController.generateFlashcards(...) receives the request.
2. LLMFacade.generateFlashcards(...) is called.
3. PromptBuilder adds the content and requested number of cards.
4. GeminiProxy sends the prompt to Gemini.
5. ResponseParser.parseFlashcards() processes the result.
6. LearningActivityFactory.createFlashcards() creates the cards.
7. Duplicate cards are removed.
8. StudyRepository.saveFlashcards() stores them.
9. The cards are displayed.

Error/Alternative Cases:

- Insufficient content displays an error.
- Invalid AI responses are retried.
- Fewer cards may be displayed if Gemini cannot generate the requested number.

F08 – Quiz Generation

Description:
The agent generates practice quizzes from selected course material.

User Interaction (GUI):
The student chooses material, quiz type, and difficulty.

Input:

material : Material
quizType : QuizType
difficulty : Difficulty

Output:
A Quiz.

AI Involvement:
AI based, with deterministic adaptive difficulty support.

Expected Workflow:

1. The student selects quiz settings.
2. If Adaptive is selected, the system follows the adaptive difficulty flow described in F12. Otherwise, the selected Easy, Medium, or Hard difficulty is used.
3. LLMFacade.generateQuiz(...) is called.
4. PromptBuilder adds material, type, and difficulty.
5. GeminiProxy sends the prompt to Gemini.
6. ResponseParser.parseQuiz() creates question data.
7. Invalid and duplicate questions are removed.
8. LearningActivityFactory.createQuiz() creates the Quiz.
9. StudyRepository.saveQuiz(quiz) stores the generated quiz.
10. The quiz is shown to the student.

Error/Alternative Cases:

- Short materials may produce fewer questions.
- Invalid AI responses are retried.
- Adaptive mode defaults to Medium if there is insufficient history.

F09 – Automatic Quiz Grading

Description:
The system grades completed quizzes and stores the result.

User Interaction (GUI):
The student clicks Submit Quiz.

Input:

quiz : Quiz
answers : Map<Question, String>

Output:
A QuizResult.

AI Involvement:
Hybrid.

Expected Workflow:

1. StudyController.submitQuiz(...) receives the quiz.
2. QuizGrader.grade(quiz, answers) begins grading.
3. Multiple-choice and true/false answers are graded deterministically.
4. Short-answer questions may use LLMFacade.evaluateAnswer().
5. PromptBuilder constructs the evaluation prompt when AI grading is required.
6. GeminiProxy sends the request to Gemini.
7. QuizGrader creates a QuestionResult for each question, recording the student's answer, awarded points, maximum points, grading status, and feedback.
8. QuizGrader combines the individual question results into a QuizResult and calculates the overall score.
9. ResultRepository.save(result) stores the QuizResult.
10. The final result and individual question outcomes are displayed.

Error/Alternative Cases:

- If Gemini is unavailable after the allowed retry, short-answer questions that require AI grading are marked Pending Review instead of being assigned a score of zero.
- Multiple-choice and true/false questions are still graded normally.
- Pending questions are excluded from finalized performance calculations until they have been graded.

F10 – Answer Explanation

Description:
The agent explains why a quiz answer is correct.

User Interaction (GUI):
The student clicks Explain beside a question.

Input:

question : Question
studentAnswer : String

Output:
An explanation string.

AI Involvement:
AI based.

Expected Workflow:

1. StudyController.explain(question, studentAnswer) receives the request.
2. LLMFacade.explain(...) is called.
3. PromptBuilder adds the question, correct answer, student's answer, and explanation instructions.
4. A Prompt is created.
5. GeminiProxy.generate(prompt) handles the request.
6. Gemini returns an explanation.
7. The explanation is displayed.

Error/Alternative Cases:

- Incomplete questions are rejected.
- GeminiProxy can return a cached explanation when available.
- Failed or rate-limited requests display an appropriate message.

F11 – Weak-Topic Identification

Description:
The system analyzes previous quiz results and identifies topics where the student performs poorly.

User Interaction (GUI):
The student opens the Insights section.

Input:
courseId : String

Output:
Map<String, Double> containing weak topic names and their average scores.

AI Involvement:
Deterministic.

Expected Workflow:

1. StudyController.viewWeakTopics(courseId) receives the request.
2. ProgressAnalyzer.getWeakTopics(courseId) is called.
3. ProgressAnalyzer retrieves quiz history from ResultRepository.
4. Results are grouped by topic.

5. Average performance is calculated.
6. Topics below the threshold are identified as weak.
7. Weak topics are displayed.

Error/Alternative Cases:

- If no quiz history exists, the student is asked to complete a quiz first.
- Topics with insufficient data are not classified.
- If no weak topics exist, the student is informed.

F12 – Adaptive Question Difficulty

Description:
The system automatically chooses quiz difficulty based on recent performance.

User Interaction (GUI):
The student selects Adaptive difficulty.

Input:

courseId : String
material : Material

Output:
A Quiz generated at Easy, Medium, or Hard difficulty.

AI Involvement:
Hybrid.

Expected Workflow:

1. The student selects Adaptive.
2. ProgressAnalyzer.computeDifficulty(courseId) is called.
3. Quiz results are retrieved from ResultRepository.
4. Recent performance is calculated.
5. The difficulty is selected.
6. The selected level is passed to LLMFacade.generateQuiz().
7. PromptBuilder builds the quiz prompt.
8. GeminiProxy sends it to Gemini.
9. ResponseParser.parseQuiz() processes the response.
10. LearningActivityFactory.createQuiz() creates the quiz.
11. The quiz is displayed.

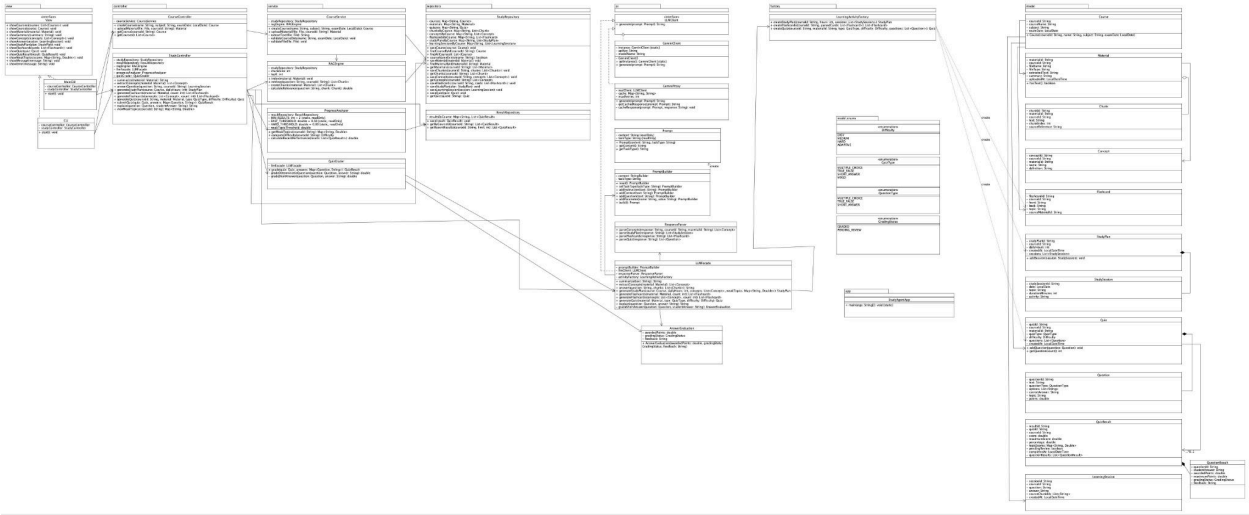
Difficulty Rule:

Below 50% → Easy
50% to 80% → Medium
Above 80% → Hard

Error/Alternative Cases:

- Fewer than two previous results is Medium.
- Invalid results are ignored.
- If generation fails, the nearest difficulty can be tried.

Task 2.1 – Class Diagram



Pattern 1 – MVC

Design Problem

The application must support both GUI and CLI interaction without mixing interface code with application logic and data management code.

Participating Classes

View, MainGUI, CLI, CourseController, StudyController, services, repositories, and model classes.

Roles

MainGUI and CLI act as views. CourseController and StudyController receive user requests and coordinate the model and services. Model and service classes perform the actual application work.

Why This Pattern Is Appropriate

Using MVC allows both the GUI and CLI to access the same application functions. So we do not have to write the course management and study logic separately for each interface.

Without This Pattern

GUI and CLI classes could contain duplicated application logic, making the project harder to maintain.

Pattern 2 – Builder

Design Problem

Different AI features require prompts containing different combinations of instructions, course content, questions, retrieved context, difficulty, and other parameters.

Participating Classes

PromptBuilder, Prompt, LLMFacade

Roles

PromptBuilder constructs the prompt step by step.

Prompt stores the completed request.

LLMFacade uses the builder to construct the prompt required by each AI feature.

Why This Pattern Is Appropriate

Each AI feature needs a different type of prompt. For example, answering a question requires the student's question and relevant course content, while generating a quiz requires the material, question type, and difficulty. The Builder pattern helps organize how these prompts are created.

Without This Pattern

Prompt construction logic would be duplicated across many AI operations.

Pattern 3 – Facade

Design Problem

AI operations involve prompt construction, API communication, response processing, and sometimes parsing structured output.

Participating Classes

LLMFacade, PromptBuilder, Prompt, LLMClient, GeminiProxy, GeminiClient, ResponseParser

Roles

LLMFacade provides simple operations such as summarize(), generateQuiz(), and answer() while hiding the internal AI subsystem.

Why This Pattern Is Appropriate

With LLMFacade, StudyController only needs to call the appropriate AI method. It does not need to handle prompt creation, communication with Gemini, or processing the response directly.

Without This Pattern

Controllers would need to directly communicate with several AI classes, increasing coupling and duplicated code.

Pattern 4 – Singleton

Design Problem

The application should not create a new Gemini API client every time an AI operation occurs.

Participating Class

GeminiClient

Role

GeminiClient contains a private constructor, one shared instance, and a public getInstance() method.

Why This Pattern Is Appropriate

All AI operations use the same Gemini configuration and API connection.

Without This Pattern

Without Singleton, different parts of the application might create their own GeminiClient objects. This would make it harder to manage the API configuration in one place.

Pattern 5 – Proxy

Design Problem

The application needs controlled access to the external Gemini service and should not duplicate retry or caching logic in every feature.

Participating Classes

LLMClient, GeminiProxy, GeminiClient, LLMFacade

Roles

Both GeminiProxy and GeminiClient implement LLMClient.

GeminiProxy receives AI requests first and can perform retries, check cached responses, or forward the request to GeminiClient.

GeminiClient performs the real Gemini API communication.

Why This Pattern Is Appropriate

API related behaviour can be handled in one place without changing each study feature.

Without This Pattern

Retry and caching logic would need to be repeated throughout the AI subsystem.

Additional Pattern – Factory

Design Problem

Quiz, flashcard, and study plan creation should not be spread throughout StudyController.

Participating Classes

LearningActivityFactory, Quiz, Question, Flashcard, StudyPlan, StudySession

Roles

LearningActivityFactory creates final learning objects after the AI response has been parsed.

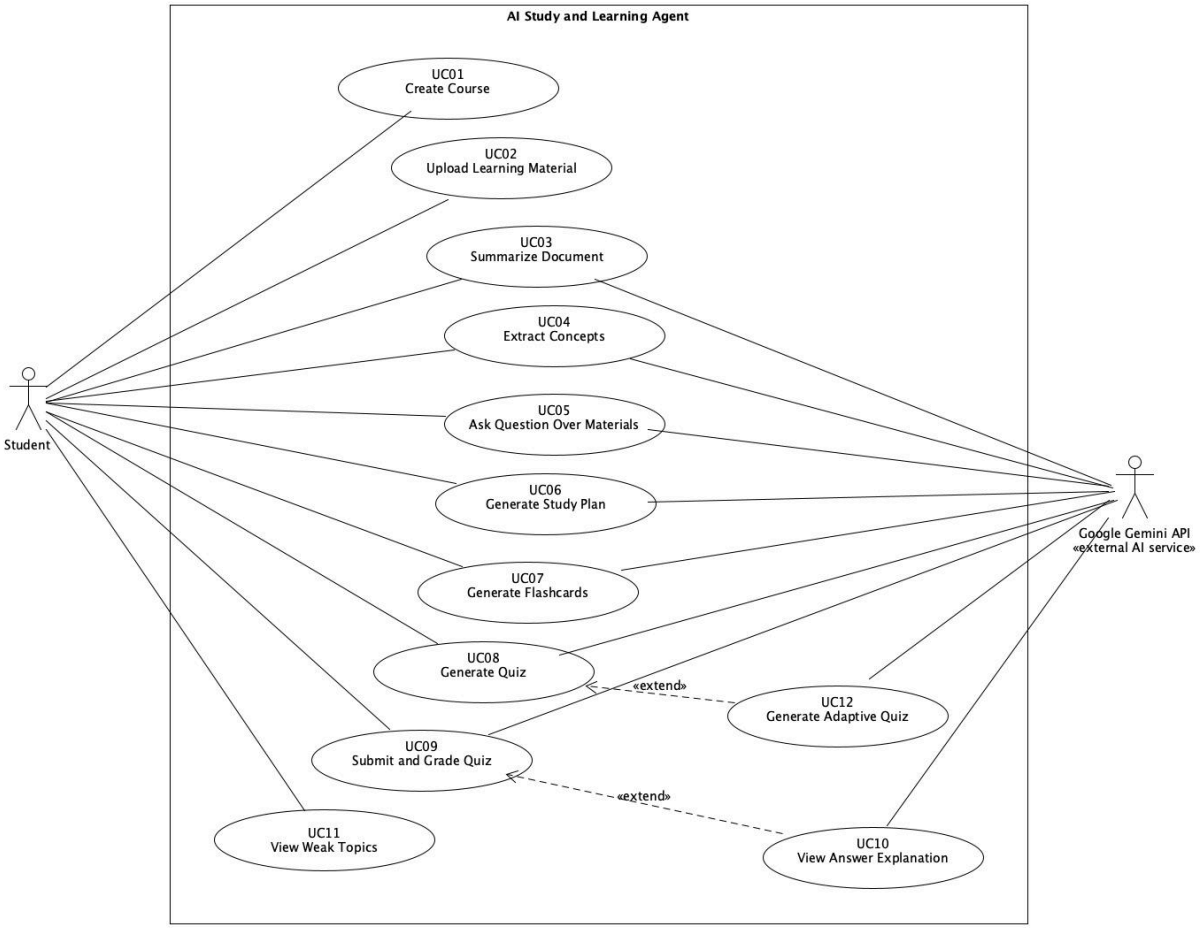
Why This Pattern Is Appropriate

Creation logic is centralized rather than duplicated in controllers.

Without This Pattern

StudyController would become responsible for constructing several different learning object types.

Task 2.2 – Use Case Diagram and



Descriptions

Actors

Student: Primary actor who uses all major study functionality.

Google Gemini API: External AI service used for AI-based and hybrid features.

There is no administrator because the application is a single user study tool. Local storage is part of the application and is therefore not modeled as an actor.

Relationships

UC10 View Answer Explanation «extend» UC09 Submit and Grade Quiz

Extension point: **Results Displayed**

UC12 Generate Adaptive Quiz «extend» UC08 Generate Quiz

Extension point: **Difficulty Selection**

UC01 – Create Course

Use Case ID: UC01
Use Case Name: Create Course

Actor(s): Student
Goal: Create a course workspace.
Preconditions: Application is running.
Trigger: Student selects New Course.

Main Success Scenario:

1. Student opens the course form.
2. Student enter course information.
3. System validates the information.
4. System creates and stores the course.
5. Course is displayed.

Alternative Flows: Invalid names, duplicate courses, past exam dates, or storage failures display an error.

Postconditions: Course is available for study activities.

Related Feature: F01

UC02 – Upload Learning Material

Use Case ID: UC02
Use Case Name: Upload Learning Material

Actor(s): Student
Goal: Add course material.
Preconditions: A course exists.
Trigger: Student selects Upload Material.

Main Success Scenario:

1. Student selects a file.
2. System validates the file.
3. Text is extracted.
4. Material is stored.
5. Material is indexed.
6. Material appears in the course.

Alternative Flows: Unsupported, oversized, protected, or unreadable files are rejected.

Postconditions: Material is stored and indexed.

Related Feature: F02

UC03 – Summarize Document

Use Case ID: UC03
Use Case Name: Summarize Document

Actor(s): Student, Google Gemini API
Goal: Generate a summary.
Preconditions: Material exists.
Trigger: Student clicks Summarize.

Main Success Scenario:

1. Student selects material.
2. System retrieves its text.
3. System constructs an AI request.
4. Gemini generates a summary.
5. Summary is stored and displayed.

Alternative Flows: Missing text, oversized documents, or AI failures are handled appropriately.

Postconditions: Summary is available.

Related Feature: F03

UC04 – Extract Concepts

Use Case ID: UC04
Use Case Name: Extract Concepts

Actor(s): Student, Google Gemini API
Goal: Extract important concepts.
Preconditions: Material exists.
Trigger: Student clicks Extract Concepts.

Main Success Scenario:

1. Material text is retrieved.
2. Concept request is sent to Gemini.
3. Gemini returns structured concepts.
4. Response is parsed.
5. Concepts are stored and displayed.

Alternative Flows: Insufficient content or invalid responses generate an error or retry.

Postconditions: Concepts are available.

Related Feature: F04

UC05 – Ask Question Over Materials

Use Case ID: UC05
Use Case Name: Ask Question Over Materials

Actor(s): Student, Google Gemini API
Goal: Receive an answer grounded in course materials.

Preconditions: Indexed course material exists.

Trigger: Student submits a question.

Main Success Scenario:

1. Student submits a question.
2. Relevant document chunks are retrieved.
3. System constructs a grounded prompt.
4. Gemini generates the answer.
5. Answer and sources are displayed.
6. Interaction is stored.

Alternative Flows: Empty questions or missing relevant material prevent the AI request.

Postconditions: Grounded answer is displayed.

Related Feature: F05

UC06 – Generate Study Plan

Use Case ID: UC06
Use Case Name: Generate Study Plan

Actor(s): Student, Google Gemini API
Goal: Generate a personalized study schedule.

Preconditions: course has material and a valid exam date.

Trigger: Student selects Generate Study Plan.

Main Success Scenario:

1. Student enters available study time.
2. System retrieves concepts and weak topics.
3. System sends information to Gemini.
4. Gemini returns a schedule.
5. Study plan is created, stored, and displayed.

Alternative Flows: Invalid study time, expired exam dates, or missing history are handled.

Postconditions: Study plan exists.

Related Feature: F06

UC07 – Generate Flashcards

Use Case ID: UC07
Use Case Name: Generate Flashcards

Actor(s): Student, Google Gemini API
Goal: Generate flashcards.

Preconditions: Material or concepts exist.

Trigger: Student selects Generate Flashcards.

Main Success Scenario:

1. Student chooses source material and card count.
2. Request is sent to Gemini.
3. Gemini returns flashcard information.
4. Flashcards are created.
5. Cards are stored and displayed.

Alternative Flows: Insufficient material or invalid responses are handled.

Postconditions: Flashcards are available.

Related Feature: F07

UC08 – Generate Quiz

Use Case ID: UC08
Use Case Name: Generate Quiz

Actor(s): Student, Google Gemini API
Goal: Generate a practice quiz.

Preconditions: Material exists.

Trigger: Student selects Generate Quiz.

Main Success Scenario:

1. Student chooses material, type, and difficulty.
2. System sends the request to Gemini.
3. Gemini generates questions.
4. Questions are parsed and validated.
5. Quiz is created and displayed.

Extension Point: Difficulty Selection

Alternative Flow: Selecting Adaptive activates UC12.

Postconditions: Quiz is available.

Related Feature: F08

UC09 – Submit and Grade Quiz

Use Case ID: UC09
Use Case Name: Submit and Grade Quiz

Actor(s): Student, Google Gemini API
Goal: Grade a completed quiz.

Preconditions: Student has completed a quiz.

Trigger: Student selects Submit Quiz.

Main Success Scenario:

1. System grades deterministic questions.
2. Short answers are evaluated by Gemini when required.
3. Final score is calculated.
4. QuizResult is stored.
5. Results are displayed.

Extension Point: Results Displayed

Alternative Flows: AI-unavailable short answers can be marked Pending Review.

Postconditions: Quiz result is stored.

Related Feature: F09

UC10 – View Answer Explanation

Use Case ID: UC10
Use Case Name: View Answer Explanation

Actor(s): Student, Google Gemini API
Goal: Understand a quiz answer.

Preconditions: Quiz results are displayed.

Trigger: Student clicks Explain.

Main Success Scenario:

1. System collects the question and the student's answer.
2. Explanation request is sent to Gemini.
3. Explanation is returned and displayed.

Alternative Flows: Cached explanations may be used if available.

Postconditions: Explanation is displayed.

Related Feature: F10

Relationship: UC10 «extend» UC09

UC11 – View Weak Topics

Use Case ID: UC11
Use Case Name: View Weak Topics

Actor(s): Student
Goal: Identify topics requiring more study.

Preconditions: Course exists.

Trigger: Student opens Insights.

Main Success Scenario:

1. Quiz results are loaded.
2. Results are grouped by topic.
3. Average performance is calculated.
4. Weak topics are identified.
5. Weak topics are displayed.

Alternative Flows: Without sufficient quiz history, the student is informed.

Postconditions: Weak-topic information is displayed.

Related Feature: F11

UC12 – Generate Adaptive Quiz

Use Case ID: UC12
Use Case Name: Generate adaptive Quiz

Actor(s): Student, Google Gemini API
Goal: Generate a quiz using automatically selected difficulty.

Preconditions: Student is generating a quiz.

Trigger: Student selects Adaptive.

Main Success Scenario:

1. Previous quiz results are loaded.
2. Recent average performance is calculated.
3. Easy, Medium, or Hard is selected.
4. Quiz generation continues using the selected difficulty.
5. Quiz is displayed.

Alternative Flows: Insufficient history defaults to Medium.

Postconditions: Adaptive quiz is generated.

Related Feature: F12

Relationship: UC12 «extend» UC08

Task 2.3 – Sequence Diagrams

The sequence diagrams show how objects collaborate at runtime. Each sequence starts with the Student, who interacts with MainGUI. The GUI forwards requests to either CourseController or StudyController.

AI requests pass through LLMFacade, PromptBuilder, GeminiProxy, and GeminiClient. Structured responses are processed through ResponseParser when necessary.

SD1 – Course Setup (F01, F02)

For course creation:

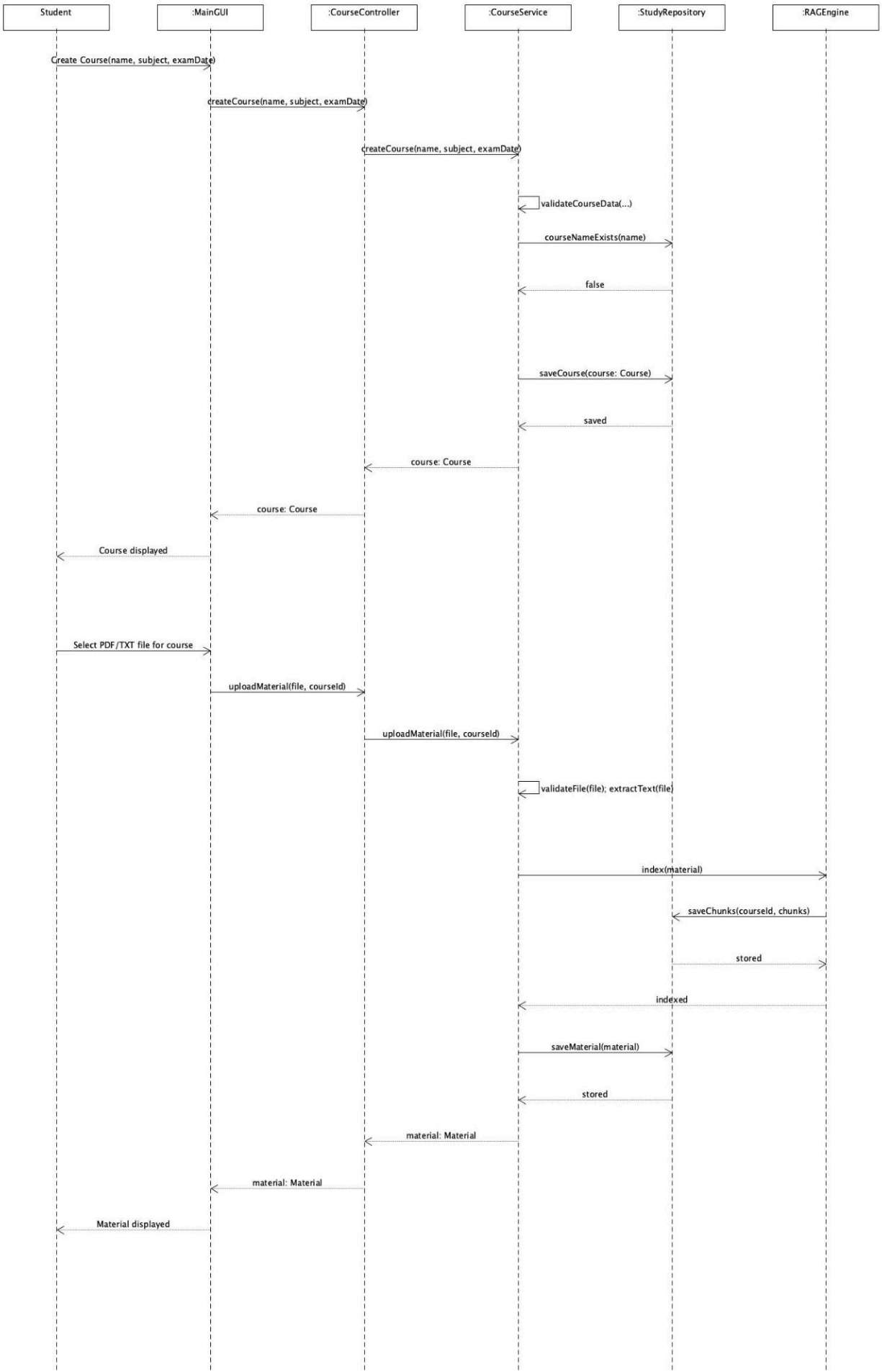
Student / MainGUI / CourseController /CourseService / StudyRepository

CourseService.createCourse() validates the input, creates a Course, and stores it through StudyRepository.

For material upload:

Student / MainGUI / CourseController / CourseService /StudyRepository /RAGEngine

CourseService validates and extracts the file text, stores the Material, and calls RAGEngine.index().


Student
:MainGUI
:CourseController
:CourseService
:StudyRepository
:RAGEngine
Create Course(name, subject, examDate)
createCourse(name, subject, examDate)
createCourse(name, subject, examDate)
validateCourseData(...)
courseNameExists(name)
false
saveCourse(course: Course)
saved
course: Course
course: Course
Course displayed
Select PDF/TXT file for course
uploadMaterial(file, courseId)
uploadMaterial(file, courseId)
validateFile(file); extractText(file)
index(material)
saveChunks(courseId, chunks)
stored
indexed
saveMaterial(material)
stored
material: Material
material: Material
Material displayed

SD2 – AI Text Processing with Builder (F03, F04, F10)

General flow:

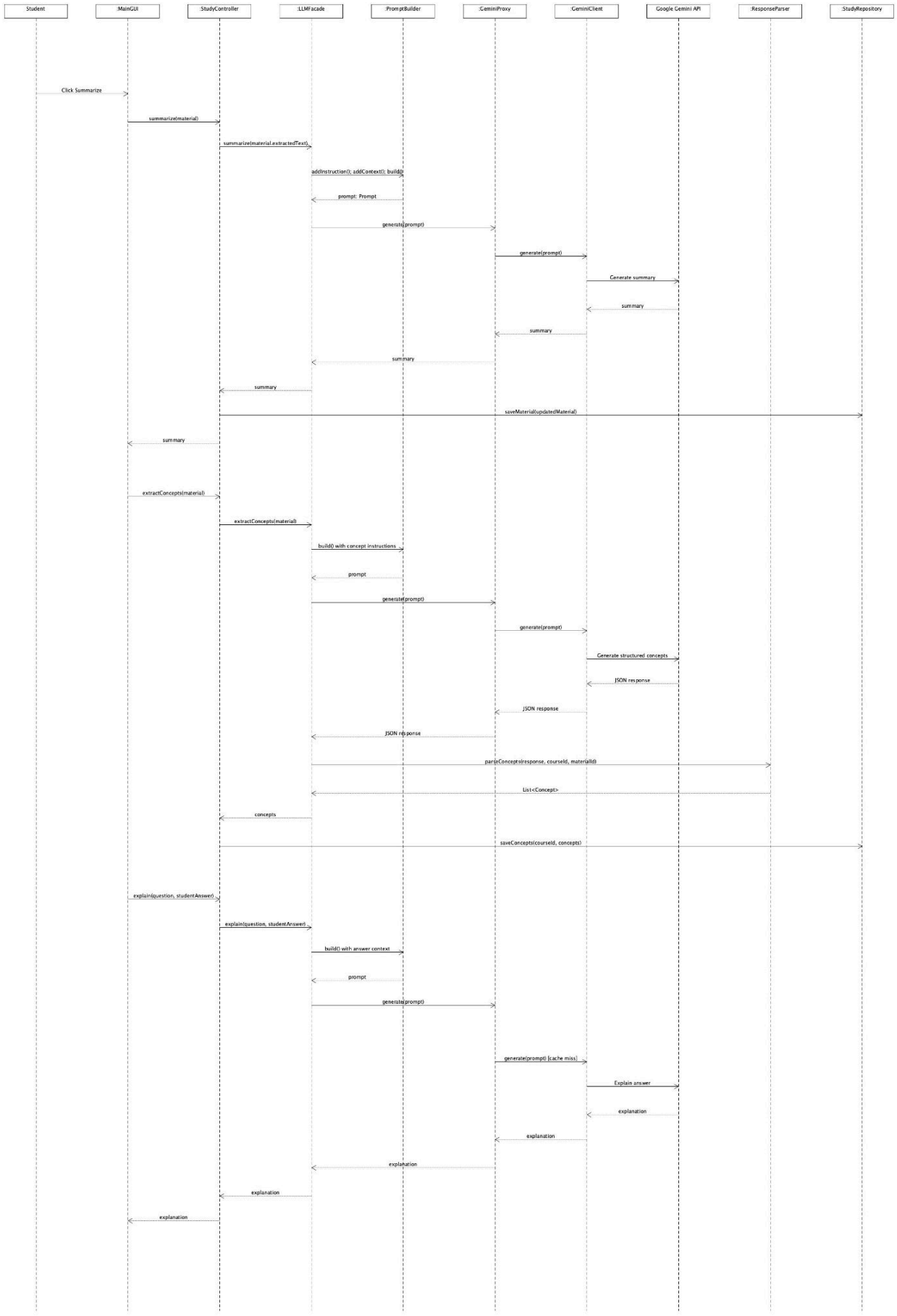
Student / MainGUI / StudyController / LLMFacade / PromptBuilder / Prompt / GeminiProxy /GeminiClient / Google Gemini API

For F03, the summary is stored using StudyRepository.

For F04, the returned response goes through:

ResponseParser / StudyRepository

For F10, the explanation is returned directly to the interface and may be cached by GeminiProxy.


Student
:MainGUI
:StudyController
:LLMFacade
:PromptBuilder
:GeminiProxy
:GeminiClient
Google Gemini API
:ResponseParser
:StudyRepository
Click Summarize
summarize(material)
summarize(material.extractedText)
addInstruction(); addContext(); build()
prompt: Prompt
generate(prompt)
generate(prompt)
Generate summary
summary
summary
summary
summary
saveMaterial(updatedMaterial)
summary
extractConcepts(material)
extractConcepts(material)
build() with concept instructions
prompt
generate(prompt)
generate(prompt)
Generate structured concepts
JSON response
JSON response
JSON response
parseConceptsResponse, courseId, materialId)
List<Concept>
concepts
saveConcepts(courseId, concepts)
explain(question, studentAnswer)
explain(question, studentAnswer)
build() with answer context
prompt
generate(prompt)
generate(prompt) [cache miss]
Explain answer
explanation
explanation
explanation
explanation
explanation

SD3 – Ask Question with RAG (F05)

Flow:

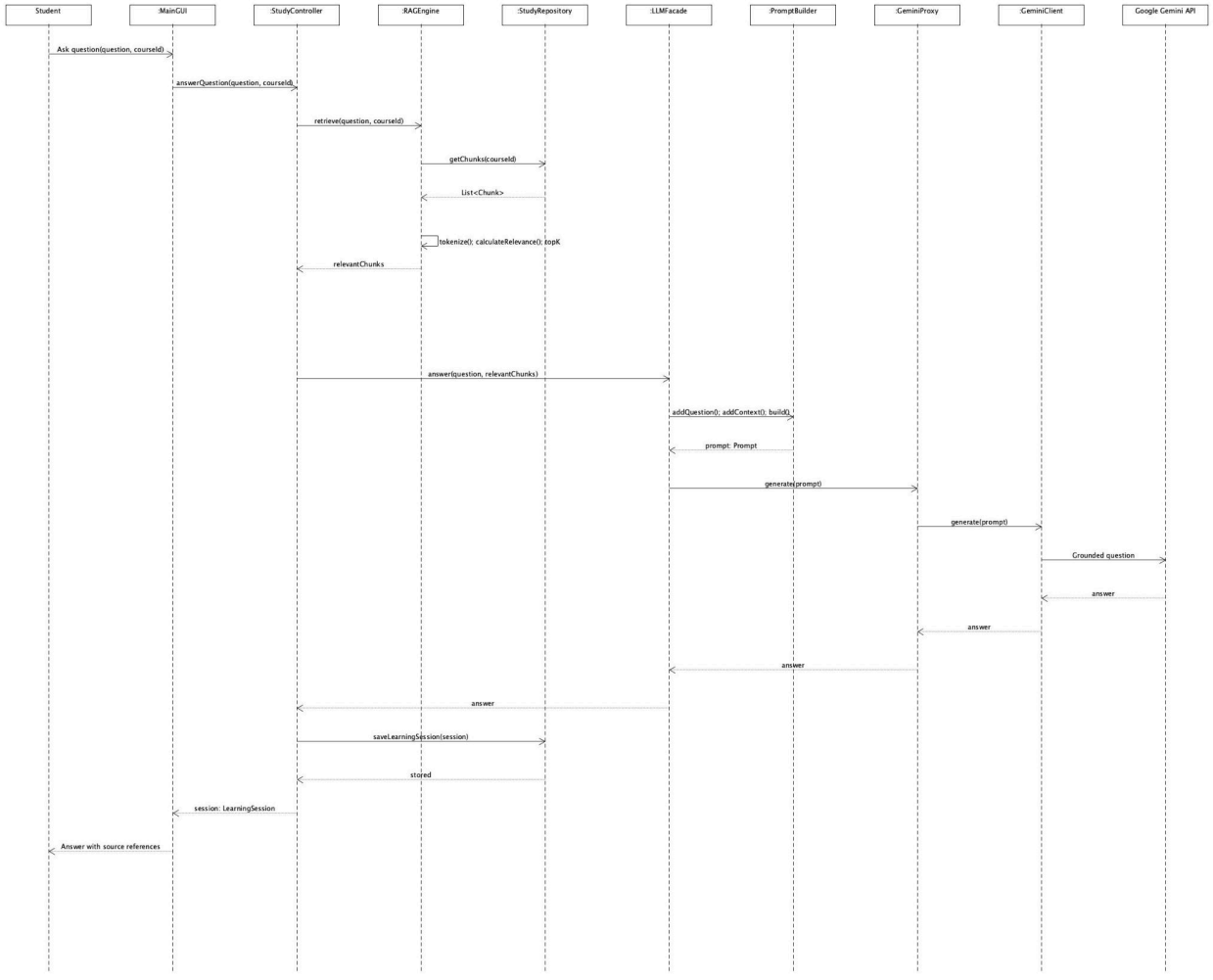
Student /MainGUI /StudyController / RAGEngine

RAGEngine.retrieve() returns relevant chunks.

Then:

StudyController / LLMFacade / PromptBuilder /GeminiProxy / GeminiClient / Google Gemini API

The returned answer is displayed and the interaction is stored through StudyRepository.saveLearningSession().



SD4 - AI Generation with Builder and Factory (F06, F07, F08, F12)

General flow:

Student /MainGUI/ StudyController

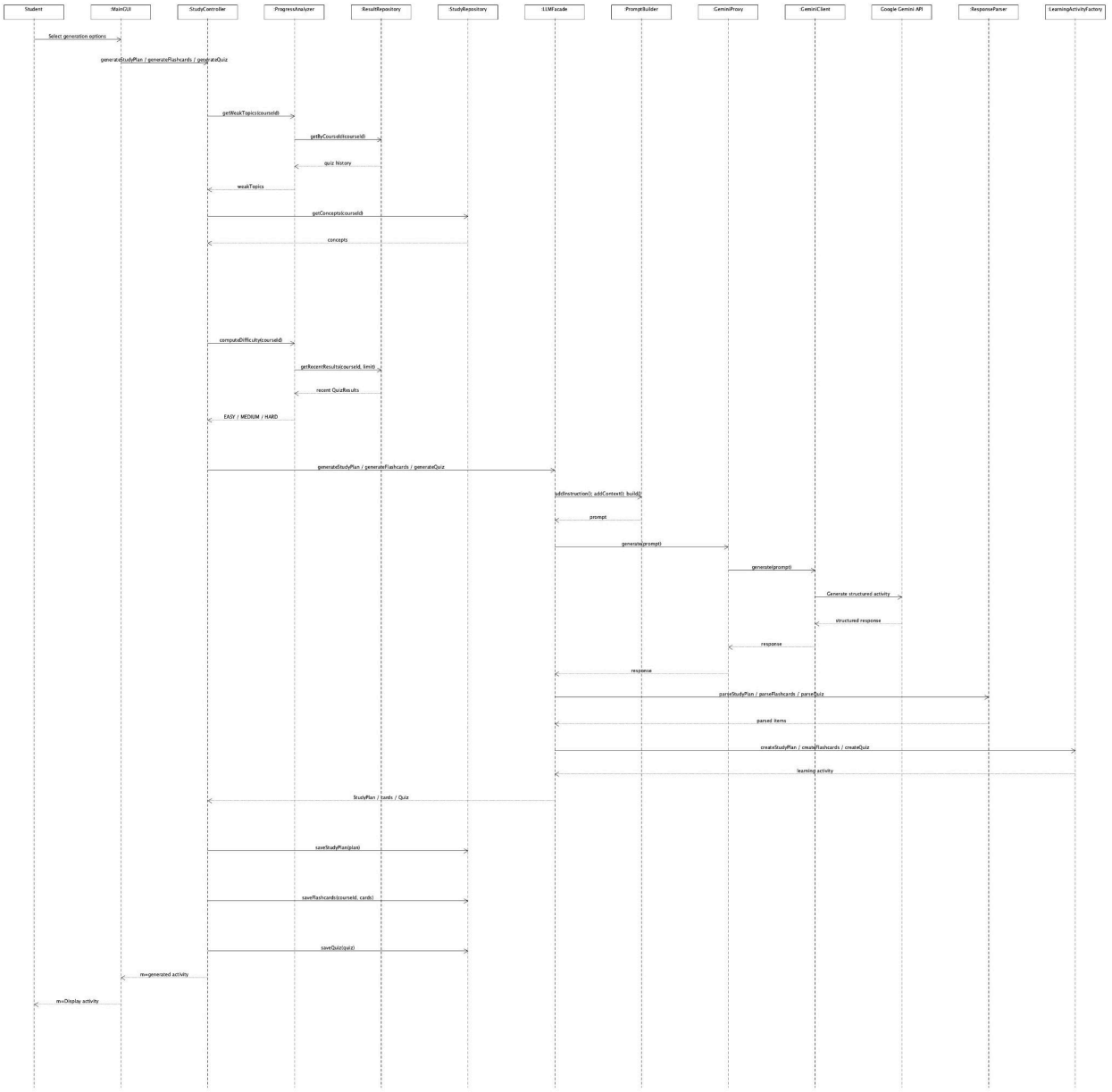
For study plans, StudyController gets concepts from StudyRepository and weak topics from ProgressAnalyzer.

For adaptive quizzes, ProgressAnalyzer.computeDifficulty() first reads results from ResultRepository.

AI generation then follows:

LLMFacade /PromptBuilder /Prompt /GeminiProxy /GeminiClient/ Google Gemini API / ResponseParser /LearningActivityFactory

The resulting study plan or flashcards are stored through StudyRepository.



SD5 – Submit and Grade Quiz (F09)

The student submits a completed quiz through MainGUI. MainGUI sends the request to StudyController, which calls QuizGrader to grade the answers.

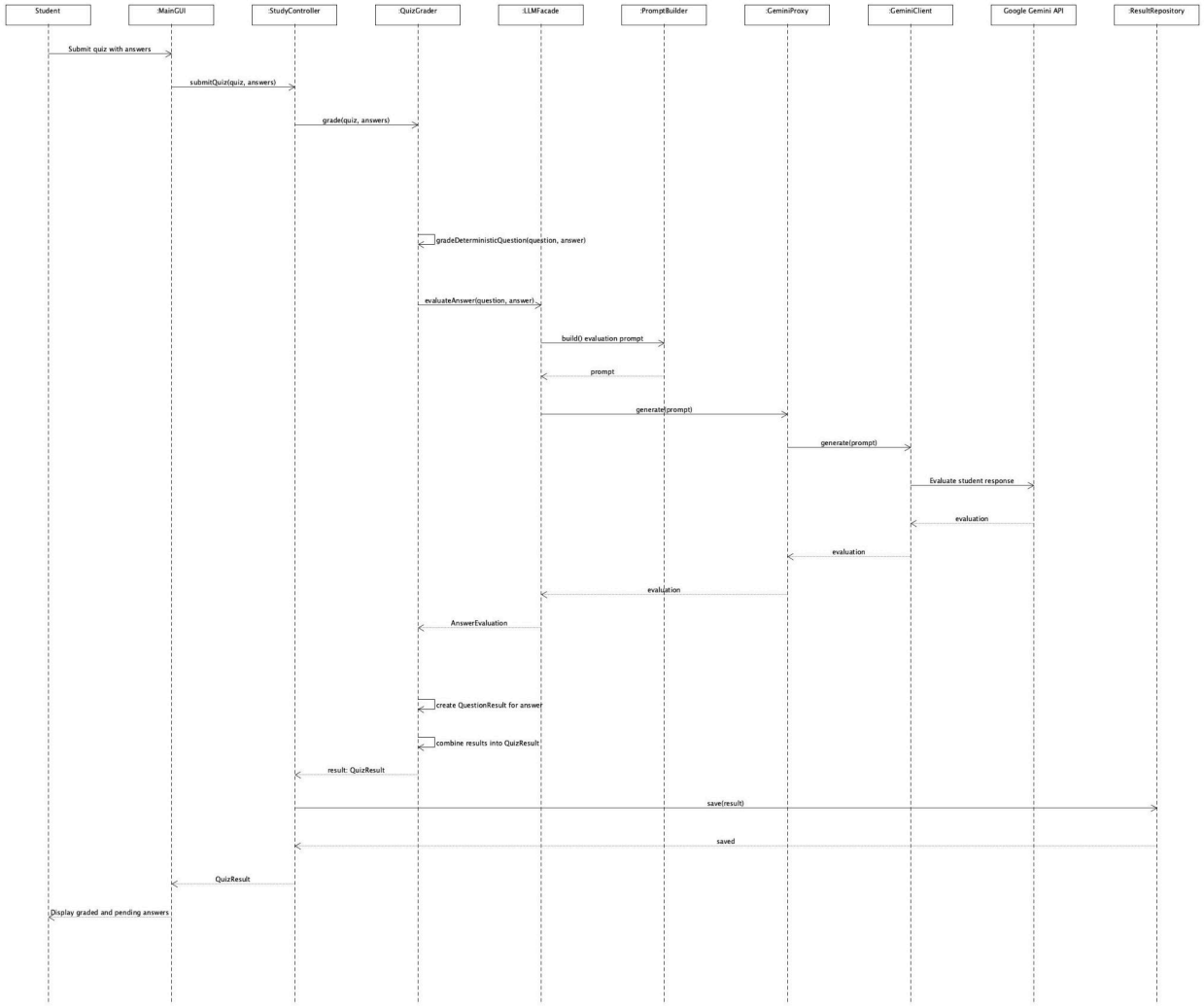
QuizGrader checks multiple-choice and true/false questions directly. For short-answer questions, QuizGrader uses LLMFacade to request an evaluation from Gemini when needed.

After evaluating each question, QuizGrader creates a QuestionResult containing the student's answer, awarded points, maximum points, grading status, and feedback.

QuizGrader then combines all QuestionResult objects into a QuizResult containing the overall score and individual question outcomes.

The completed QuizResult is returned to StudyController, which saves it using ResultRepository.save(result). Finally, the result is returned to MainGUI and displayed to the student.

If Gemini is unavailable after the allowed retry, LLMFacade returns an AnswerEvaluation with a Pending Review status. QuizGrader records this status in the corresponding QuestionResult without treating the answer as incorrect. The system still grades multiple-choice and true/false questions normally. The QuizResult is saved with a pending-review indicator, and the student is informed that some answers have not yet been graded.



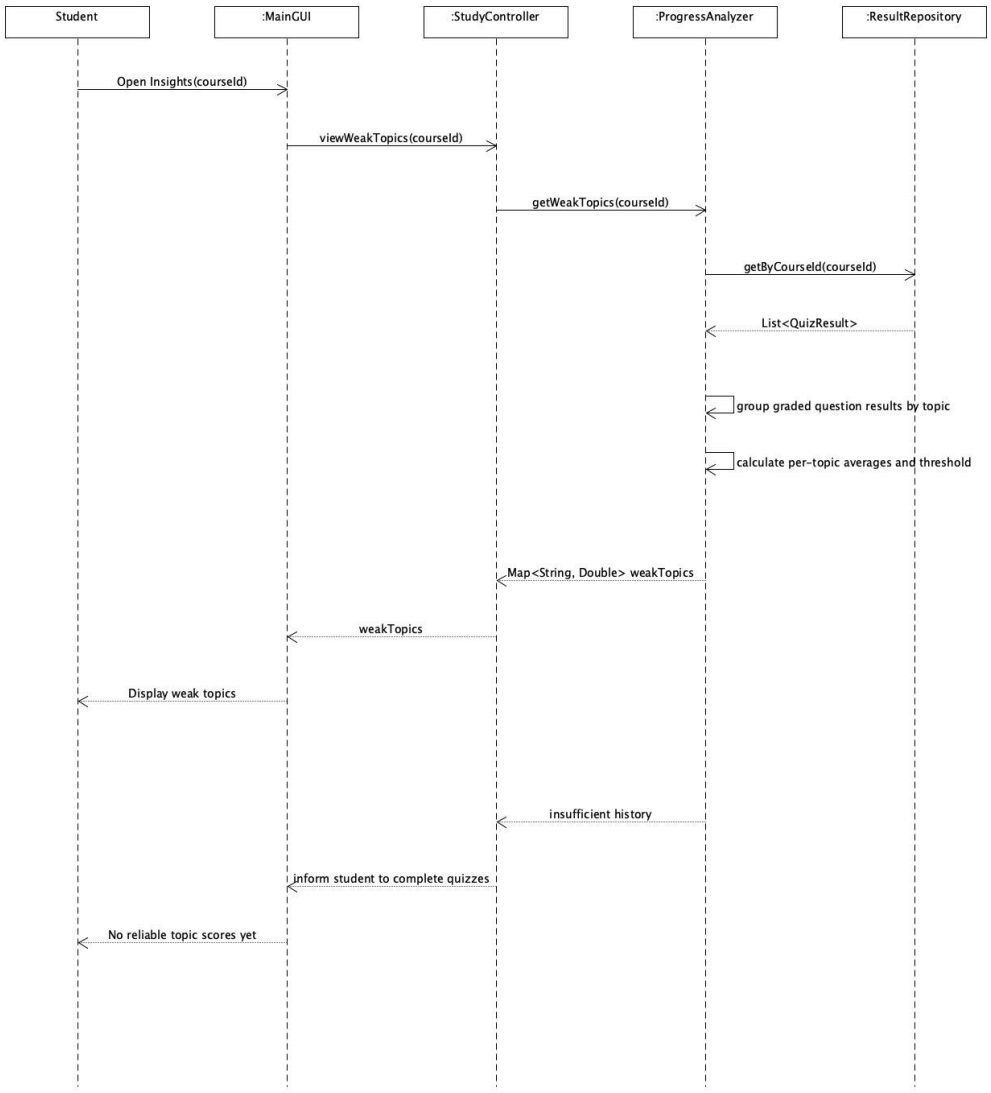
SD6 – View Weak Topics (F11)

Flow:

Student / MainGUI /StudyController /ProgressAnalyzer /ResultRepository

ResultRepository returns quiz history.

ProgressAnalyzer groups results by topic, calculates average scores, and returns the weak-topic information. The results are displayed through MainGUI.



Task 3 – Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence	Design Patterns
F01	Create and save a course	Deterministic	UC01 – Create Course	CourseController, CourseService, Course, StudyRepository	createCourse(), saveCourse()	SD1	MVC
F02	Upload PDF/TXT material, extract its text, save it, and index it for retrieval	Deterministic	UC02 – Upload Learning Material	MainGUI/CLI, CourseController, CourseService, Material, StudyRepository, RAGEngine, Chunk	uploadMaterial(), extractText(), saveMaterial(), index()	SD1	MVC

						Diagram	
F03	Generate and save a document summary	AI based	UC03 – Summarize Document	StudyController, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, Material, StudyRepository	summarize(), build(), generate(), saveMaterial()	SD2	MVC, Builder, Facade, Proxy, Singleton
F04	Extract important concepts and definitions from course material	AI based	UC04 – Extract Concepts	StudyController, LLMFacade, PromptBuilder, GeminiProxy, GeminiClient, ResponseParser, Concept, StudyRepository	extractConcepts(), build(), generate(), parseConcepts(), saveConcepts()	SD2	MVC, Builder, Facade, Proxy, Singleton
F05	Answer questions using relevant chunks retrieved from uploaded course materials	Hybrid	UC05 – Ask Question	StudyController, RAGEngine, Chunk, LLMFacade, PromptBuilder, GeminiProxy, GeminiClient, StudyRepository	retrieve(), answer() , addQuestion(), addContext, build(), generate(), saveLearningSession()	SD3	MVC, Builder, Facade, Proxy, Singleton
F06	Generate a personalized study plan using exam date, study time, concepts, and weak topics	Hybrid	UC06 – Generate Study Plan	StudyController, ProgressAnalyzer, StudyRepository, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, ResponseParser, LearningActivityFactory, StudyPlan	getWeakTopics(), getConcepts(), generateStudyPlan(), parseStudyPlan(), createStudyPlan()	SD4	MVC, Builder, Facade, Proxy, Singleton, Factory
F07	Generate and save flashcards from material or extracted concepts	AI based	UC07 – Generate Flashcards	StudyController, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, ResponseParser, LearningActivityFactory, Flashcard, StudyRepository	generateFlashcards(), build(), parseFlashcards(), createFlashcards(), saveFlashcards()	SD4	MVC, Builder, Facade, Proxy, Singleton, Factory

F08	Generate a practice quiz using selected quiz type and difficulty	AI based	UC08 – Generate Quiz	StudyController, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, ResponseParser, LearningActivityFactory, Quiz, Question,StudyRepository	generateQuiz(), build(), parseQuiz(), createQuiz(),saveQuiz()	SD4	MVC, Builder, Facade, Proxy, Singleton, Factory
F09	Grade a completed quiz and save its result.	Hybrid	UC09 – Submit and Grade Quiz	StudyController, QuizGrader, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, QuizResult, ResultRepository	submitQuiz(), grade(), evaluateAnswer(), generate(), save()	SD5	MVC, Builder, Facade, Proxy, Singleton
F10	Generate an answer explanation	AI based	UC10 – View Answer Explanation	StudyController, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, Question	explain(), build(), generate()	SD2	MVC, Builder, Facade, Proxy, Singleton
F11	Identify weak topics from quiz history	Deterministic	UC11 – View Weak Topics	StudyController, ProgressAnalyzer, ResultRepository, QuizResult	getWeakTopics(), getByCourseId(courseId)	SD6	MVC
F12	Select adaptive difficulty and generate a quiz	Hybrid	UC12 – Generate Adaptive Quiz	StudyController, ProgressAnalyzer, ResultRepository, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, ResponseParser, LearningActivityFactory, Quiz	computeDifficulty(), generateQuiz(), build(), parseQuiz(), createQuiz()	SD4	MVC, Builder, Facade, Proxy, Singleton, Factory

Task 4 – Explain How Each Feature Is Realized

F01 – Course Creation

Related Use Case: UC01 – Create Course
Related Sequence Diagram: SD1 – Course Setup

Classes involved:
CourseController, CourseService, Course, StudyRepository

Important methods:
CourseService.createCourse()
StudyRepository.saveCourse()

Execution:
When the student submits the course form, CourseController forwards the information to CourseService.createCourse(). CourseService validates the data and creates a Course. The course is stored using StudyRepository.saveCourse() and displayed to the student.

F02 – Upload Learning Materials

Related Use Case: UC02 – Upload Learning Material
Related Sequence Diagram: SD1 – Course Setup

Classes involved:
CourseController, CourseService, Material, StudyRepository, RAGEngine, Chunk

Important methods:
CourseService.uploadMaterial()
CourseService.extractText()
StudyRepository.saveMaterial()
RAGEngine.index()

Execution:
CourseService validates the uploaded file and extracts its text. A Material is created and stored through StudyRepository. RAGEngine.index() divides the material into Chunk objects so that relevant sections can later be retrieved.

F03 – Document Summarization

Related Use Case: UC03
Related Sequence Diagram: SD2

Classes involved:
StudyController, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, Material, StudyRepository

Important methods:
summarize(), build(), generate(), saveMaterial()

Execution:
StudyController calls LLMFacade.summarize(). PromptBuilder constructs the summarization prompt. GeminiProxy sends the completed prompt through GeminiClient. The returned summary is stored with the material using StudyRepository.

F04 – Concept Extraction

Related Use Case: UC04
Related Sequence Diagram: SD2

Classes involved:
StudyController, LLMFacade, PromptBuilder, GeminiProxy, GeminiClient, ResponseParser, Concept, StudyRepository

Important methods:
extractConcepts(), build(), generate(), parseConcepts(), saveConcepts()

Execution:
The material text is sent through LLMFacade. PromptBuilder builds the request, and GeminiProxy sends it to Gemini. ResponseParser converts the returned information into Concept objects, which are stored using StudyRepository.

F05 – Question Answering Over Materials

Related Use Case: UC05
Related Sequence Diagram: SD3

Classes involved:
StudyController, RAGEngine, Chunk, LLMFacade, PromptBuilder, GeminiProxy, GeminiClient, StudyRepository

Important methods:
retrieve(), answer(), addQuestion(), addContext(), build(), generate(), saveLearningSession()

Execution:
RAGEngine.retrieve() finds the most relevant chunks from uploaded course materials. LLMFacade uses PromptBuilder to combine the question with the retrieved context. Gemini generates a grounded answer. The answer and source information are displayed, and StudyRepository.saveLearningSession() records the interaction.

F06 – Personalized Study Plan Generation

Related Use Case: UC06
Related Sequence Diagram: SD4

Classes involved:
StudyController, ProgressAnalyzer, StudyRepository, LLMFacade, PromptBuilder, GeminiProxy, ResponseParser, LearningActivityFactory, StudyPlan, StudySession

Important methods:
getWeakTopics(), getConcepts(), generateStudyPlan(), parseStudyPlan(), createStudyPlan(), saveStudyPlan()

Execution:
ProgressAnalyzer determines the student's weak topics while StudyRepository provides course concepts. This information is passed to LLMFacade. Gemini generates a structured schedule. ResponseParser processes the response and LearningActivityFactory creates the final StudyPlan. The plan is then stored using StudyRepository.saveStudyPlan() and displayed to the student.

F07 – Flashcard Generation

Related Use Case: UC07
Related Sequence Diagram: SD4

Classes involved:
StudyController, LLMFacade, PromptBuilder, GeminiProxy, ResponseParser, LearningActivityFactory, Flashcard, StudyRepository

Important methods:
generateFlashcards(), build(), generate(), parseFlashcards(), createFlashcards(), saveFlashcards()

Execution:
The selected course content is passed to LLMFacade. PromptBuilder constructs the flashcard request and Gemini generates structured card information. ResponseParser processes the result, LearningActivityFactory creates the flashcards, and StudyRepository stores them.

F08 – Quiz Generation

Related Use Case: UC08
Related Sequence Diagram: SD4

Classes involved:
StudyController, LLMFacade, PromptBuilder, GeminiProxy, ResponseParser, LearningActivityFactory, Quiz, Question

Important methods:
generateQuiz(), build(), generate(), parseQuiz(), createQuiz()

Execution:
ResponseParser converts the generated response into questions, and LearningActivityFactory creates the final Quiz. StudyController then saves the quiz through StudyRepository.saveQuiz() before displaying it to the student.

F09 – Automatic Quiz Grading

Related Use Case: UC09
Related Sequence Diagram: SD5

Classes involved:
StudyController, QuizGrader, LLMFacade, PromptBuilder, GeminiProxy, GeminiClient, QuizResult, ResultRepository

Important methods:
submitQuiz(), grade(), evaluateAnswer(), build(), generate(), save()

Execution:
QuizGrader creates a QuestionResult for each question and combines them into a QuizResult containing the final score and individual grading outcomes. The result is then stored through ResultRepository and displayed to the student.

F10 – Answer Explanation

Related Use Case: UC10
Related Sequence Diagram: SD2

Classes involved:
StudyController, LLMFacade, PromptBuilder, Prompt, GeminiProxy, GeminiClient, Question

Important methods:
explain(), build(), generate()

Execution:
When the student requests an explanation, LLMFacade uses PromptBuilder to create a prompt containing the question, correct answer, and student's answer. GeminiProxy sends the request to Gemini. The generated explanation is returned to the student.

F11 – Weak-Topic Identification

Related Use Case: UC11
Related Sequence Diagram: SD6

Classes involved:
StudyController, ProgressAnalyzer, ResultRepository, QuizResult

Important methods:
getWeakTopics(), getByCourseId(courseId)

Execution:
ProgressAnalyzer.getWeakTopics() loads previous QuizResult objects from the ResultRepository. Results are grouped by topic and average performance is calculated. Topics below the configured threshold are returned and displayed as weak topics.

F12 – Adaptive Question Difficulty

Related Use Case: UC12
Related Sequence Diagram: SD4

Classes involved:
StudyController, ProgressAnalyzer, ResultRepository, LLMFacade, PromptBuilder, GeminiProxy, ResponseParser, LearningActivityFactory, Quiz

Important methods:
computeDifficulty(), getByCourseId(courseId), generateQuiz(), build(), generate(), parseQuiz(), createQuiz()

Execution:
ProgressAnalyzer.computeDifficulty() reads recent quiz results from ResultRepository and calculates the student's recent performance. It selects Easy, Medium, or Hard difficulty. The selected level is passed to the normal quiz-generation process, where Gemini generates questions and LearningActivityFactory creates the final adaptive quiz.